Loops: Deep Dive

Why the implementation looks like this

Gregory S. DeLozier, PhD

The Body Is Just a Block

`while (condition) { ... }` reuses `parse_block` from Chapter 7 completely unchanged. There was no decision to make here in the sense of choosing between options; once a loop body is required to be a brace-delimited block, it is definitionally the same grammar production an `if` branch already uses, down to the same "statement_list is optional" rule that makes `while (x) { }` legal.

```
body, tokens = parse_block(tokens)
```

That is the entire body-parsing logic in `parse_while_statement`. One line, because the real work happened in Chapter 7.

The Body Is Just a Block (continued)

The interesting fact is what reuse implies in reverse: every property a block already had, a loop body gets for free, without anyone deciding it should. Empty bodies are legal, because `{}` already was. Trailing semicolons before the closing brace are legal, because that was already true of any block. None of that was designed with loops in mind. It was inherited the moment `parse_block` was chosen instead of something new.

Reuse is not a shortcut here. It is where the correctness comes from.

break and continue Are Leaves

```
if ast["tag"] == "break":
    return None, "break"

if ast["tag"] == "continue":
    return None, "continue"
```

Two lines each, and that is the whole implementation. Neither evaluates a child. Neither inspects the environment.

break and continue Are Leaves (continued)

The decision here is more subtle than the two-line implementation suggests: `break` and `continue` do not know they are inside a loop. They do not check. They *cannot* check, because nothing about evaluating a node tells it what is calling it. They simply report a status and let whatever encloses them decide what, if anything, that status means.

This is the same design chapter 6 already committed to for `exit`: **a status is a fact about what happened, not an instruction about what to do next.** The `while` node is the first thing that happens to care what `"break"` and `"continue"` mean. Nothing else in the interpreter needs to.

Four Outcomes, Two Destinations

Every design decision in `evaluate()`'s `while` case comes down to this classification. There are exactly four ways a pass through the Python `while True:` loop can end, and only two distinct places they can go:

Outcome	Where it goes
Condition itself returns a status	Out, immediately
Condition is false	Loop ends: `(None, None)`
Body returns `"break"`	Loop ends: `(None, None)`
Body returns anything else non-`None`, non-`"continue"`	Out, immediately
Body completes normally, or returns `"continue"`	Back to the top, recheck condition

Four Outcomes, Two Destinations (continued)

Two outcomes share the exact same destination and return value despite meaning completely different things: a condition that finally came back false, and a body that explicitly asked to stop. The decision to let them converge, rather than distinguish them with a different status or return value, is what makes the code four `if` statements long instead of something with its own internal signaling scheme.

Convergence Is Correct, Not a Shortcut

```
if not condition:
    return None, None
...
if status == "break":
    return None, None
```

(Two non-adjacent lines from the same function, shown together on purpose: identical outcome, unrelated cause.)

It would be possible to return something that distinguishes “the condition failed” from “something asked to stop.” Nothing in this interpreter, or in any caller of a `while` node, has ever needed that distinction.

Convergence Is Correct (continued)

Introducing a way to tell the two apart would be solving a problem nobody has, in exchange for a status space that now needs documenting, testing, and explaining to every future chapter that touches a loop.

This is the same discipline chapter 7 already applied to `if`: **`(None, None)` is "normal completion," full stop, regardless of which of several paths produced it.** Chapter 8 did not invent that discipline. It just found its second use.

No New Code to Reject Misuse

The outline for this chapter asked for a decision: diagnose `break`/`continue` outside a loop during parsing, or at evaluation. The implementation picks neither, in the sense of writing anything new.

```
if status == "exit":
    # Keep process termination outside the recursive evaluator.
    sys.exit(value)
if status is not None:
    raise ValueError(f"Unhandled evaluation status: {status}")
```

This is the runner's fallback, written for Chapter 6, **before `break` or `continue` existed.**

No New Code to Reject Misuse (continued)

`break` reaching the runner unconsumed is not a gap that got patched after the fact. The check was never special-cased for `exit` in the first place; `is not None` was always the condition, not `== "exit"`. It is the same fallback doing the same job it was always going to do, for a status that did not exist yet when the fallback was written.

The misuse this chapter's outline worried about was already handled, two chapters before the misuse was possible.

Why This Matters Beyond Loops

The `(value, status)` contract from chapter 6 was deliberately generic: `status` is "something other than normal completion," not an enum of known cases. Chapter 6 tested this directly, with a synthetic status standing in for a future one that did not exist yet.

Chapter 8 is the first chapter to actually need that genericity, and it did not need to touch the contract at all to get it.

No change to `evaluate_source`. No change to the runner. No change to `if`, `statement_list`, or `program`.

continue Returns to the Condition, Not the Body

```
if status is not None and status != "continue":
    return value, status
# Normal completion and "continue" both return to the condition.
```

The tempting alternative is to have `continue` jump back into the body somehow, or perform an implicit update before looping. Neither is what happens.

continue Returns to the Condition (continued)

`continue` performs no update of its own because nothing about the status `"continue"` carries any information beyond "stop running this body now." Whatever state changes already happened, happened; whatever was going to happen next in the body does not.

The chapter's own example makes the consequence concrete: move the counter update to *after* the `if (m == 3) { continue; }` check instead of before it, and the loop never advances past `m = 3`. **That is not a bug in `continue`.** It is `continue` doing exactly what it is defined to do, applied to a program that no longer has anywhere for progress to come from.

Bounded Subprocess Tests, Narrowly Scoped

```
SUBPROCESS_TIMEOUT = 5
...
result = subprocess.run([sys.executable, str(runner), source],
                        capture_output=True, text=True,
                        stdin=subprocess.DEVNULL,
                        timeout=SUBPROCESS_TIMEOUT)
```

Two tests in `test_loops.py` run small Vertex programs through the runner as a real subprocess with a wall-clock timeout, specifically so a regression that breaks loop termination fails fast instead of hanging CI forever.

Bounded Subprocess Tests (continued)

An earlier draft of the chapter implied this protected the whole test suite. It does not: every other loop test calls the evaluator in-process, with no timeout at all, and a regression there would still hang.

The actual scope is narrower, and more useful to know precisely: these two tests guard the two places in the suite most likely to actually run an unbounded loop by construction, not a blanket safety net over everything.

What Carries Into Chapter 9

Chapter 9 adds a mutable, compound value. Nothing about loops changes to accommodate it: a list is indexed and assigned the same way any other value is, inside a body, inside a condition, anywhere an expression or a target is already allowed.

The `while` node itself needs zero changes.

The pattern holds: a chapter that looks like it should touch everything upstream, by design, touches nothing upstream at all.